

DOCUMENT 2: FUNCTIONAL REQUIREMENTS DOCUMENT (FRD)

Project Name: Automated Reporting System (Google Sheets & Apps Script)
Prepared By: Mukesh Rathore, Technical Project Manager
Date: July 12, 2026 Version: 1.0
Reference: Maps to BRD v1.0

1. Introduction

This Functional Requirements Document (FRD) details the specific system behaviors, data flows, and technical functionalities required to fulfill the business objectives outlined in the BRD. It serves as the primary blueprint for the Development and QA teams.

2. System Architecture & Data Flow

- Source Layer:** Multiple raw Google Sheets containing operational data.
- Processing Layer (Apps Script):** Custom scripts execute ETL operations, clean data, handle transformations, and push data to the Master Data Sheet.
- Presentation Layer:** Google Sheets Dashboard utilizing QUERY, VLOOKUP, Pivot Tables, and native charting tools to visualize Master Data.
- Automation Layer:** Time-driven Apps Script triggers generate HTML emails and dispatch them via the Gmail API.

3. Detailed Functional Requirements

3.1 Data Extraction & Transformation (ETL)

Req ID	Functional Requirement	Technical Implementation Notes
FR-1.1	The system shall read data from defined ranges in all source sheets.	Use SpreadsheetApp.openById() and getSheetValues().
FR-1.2	The system should clean data by removing blank rows and standardizing date formats.	Implement array-based JavaScript mapping to filter nulls and format dates to YYYY-MM-DD.

FR-1.3	The system shall merge data from multiple sheets into a single "Master Data" tab.	Concatenate arrays in memory and use <code>setValues()</code> to write to the master sheet in a single batch operation to optimize performance.
FR-1.4	The system shall handle duplicate records based on a unique "Transaction ID" column.	Use a JavaScript Set or Map to filter duplicates before writing to the Master sheet.

3.2 Dynamic Dashboard UI/UX

Req ID	Functional Requirement	Technical Implementation Notes
FR-2.1	The dashboard shall include a "Date Range" slicer to filter all visualizations dynamically.	Utilize native Google Sheets Slicers linked to Pivot Table data sources.
FR-2.2	The dashboard shall display a "Summary KPI" section showing Total Revenue, Total Volume, and Growth %.	Use <code>QUERY()</code> and <code>SUMIFS()</code> formulas referencing the "Master Data" tab.
FR-2.3	The dashboard shall render interactive bar charts and line graphs for trend analysis.	Use native Google Sheets Charts, configured to update automatically when the underlying Pivot Table refreshes.
FR-2.4	The dashboard shall include a "Refresh Data" custom button for manual triggering.	Create a custom UI menu or drawing-based button linked to an Apps Script <code>refreshDashboard()</code> function.

3.3 Automated Email Triggers

Req ID	Functional Requirement	Technical Implementation Notes
FR-3.1	The system shall send an automated email every Monday at 8:00 AM.	Configure a time-driven trigger (<code>ScriptApp.newTrigger('sendWeeklyReport').timeBased().onWeekDay(MONDAY).atHour(8).create()</code>).
FR-3.2	The email body shall be formatted in HTML, including a dynamic summary table.	Use <code>HtmlService.createHtmlOutputFromFile()</code> to generate the email body, injecting data via template scripts.

FR-3.3	The system shall attach the current dashboard as a PDF to the email.	Use SpreadsheetApp.getActiveSpreadsheet().getAs('application/pdf') to generate and attach the file via GmailApp.sendEmail().
---------------	--	--

3.4 Error Handling & Logging

Req ID	Functional Requirement	Technical Implementation Notes
FR-4.1	The system shall log all script executions, errors, and execution times to a hidden "Logs" sheet.	Implement a try...catch block in the main function. Log Timestamp, Function Name, Status, and Error Message.
FR-4.2	If the script encounters a fatal error, it shall send an alert email to the IT Admin.	In the catch block, trigger GmailApp.sendEmail() to the admin address with the error stack trace.
FR-4.3	The system shall prevent concurrent script executions to avoid data corruption.	Implement LockService.getScriptLock().tryLock(10000) at the beginning of the main execution function.

5. Data Dictionary & Mapping (Sample)

Source Sheet	Source Column	Target Master Sheet Column	Data Type	Transformation Logic
Sales_Sheet_1	Date	Transaction_Date	Date	Convert to ISO format (YYYY-MM-DD).
Sales_Sheet_1	Amt	Revenue_USD	Currency	Round to 2 decimal places.
Inventory_Sheet	Item_Code	SKU_ID	String	Convert to uppercase, trim whitespace.

5. Non-Functional Requirements (Technical Constraints)

- **Performance:** The ETL script must process and load up to 50,000 rows of data within 4 minutes to respect the Google Apps Script 6-minute hard limit.
- **Optimization:** Scripts must use batch reading/writing (getValues() / setValues()) rather than cell-by-cell operations to minimize API calls.

- **Security:** No sensitive credentials (like API keys) shall be hardcoded. Use `PropertiesService.getScriptProperties()` for storing configuration variables.
- **Maintainability:** Code must be modular, well-commented, and follow standard JavaScript/Apps Script linting practices.

6. Use Cases

Use Case 1: Automated Weekly Report Generation

- **Actor:** System (Time Trigger)
- **Pre-condition:** Master data is updated; Trigger is active.
- **Flow:**
 1. Trigger fires at scheduled time.
 2. Script fetches latest aggregated data.
 3. Script generates HTML body and PDF attachment.
 4. Script sends email via Gmail API.
 5. Script logs success in the "Logs" sheet.
- **Post-condition:** Stakeholders receive the report; execution is logged.